A recursion category for Blum–Shub–Smale computation

Florian Lengyel*

October 22, 2026

Abstract

We construct a dominical recursion category whose arrows are Blum–Shub–Smale (BSS) computable partial functions over the real numbers. The objects are positive finite powers of a space of tagged finite real tuples. An explicit computable bijection identifies this space with its square, and program specialization makes a universal evaluator a Turing morphism. The category of BSS-computable partial functions between finite-dimensional real spaces embeds fully faithfully in the restriction idempotent completion.

The existence of a recursion-category presentation of BSS computation was stated in [6, §3, p. 25], with the construction omitted. We give the construction over $\mathbb{R}$. The computation model and its universal machines are due to Blum, Shub, and Smale [1, 2]; the categorical definitions are those of Di Paola and Heller [4] and Heller [5]. Vigna [7] gives another categorical characterization of BSS computability over commutative ordered rings. Our construction specifies an isotype and a Turing morphism and identifies the fixed-dimensional computations in its completion.

1 Finite data and a computable pairing

We use deterministic BSS machines over the ordered field $\mathbb{R}$, with exact arithmetic, equality and order tests, and finite programs. Each program may contain finitely many arbitrary real constants. Running time is unrestricted. Inputs and outputs are finite tuples whose lengths are recorded; machines have unbounded tape storage. These are the finite-sequence conventions used for universal computation in [1, §8] and [2, Chapter 3]. Division by zero is treated as divergence. A *partial function* $f : A \rightharpoonup B$ is a function from a subset $\mathrm{dom}(f) \subseteq A$ to B; the subset $\mathrm{dom}(f)$ is its *domain of definition*. The partial function is *total* when $\mathrm{dom}(f) = A$. It is BSS-computable when a machine halts exactly on this subset and returns $f(a)$ on each input $a \in \mathrm{dom}(f)$.

Put $\omega = \{0, 1, \ldots\}$ and $\mathbb{R}^{<\omega} = \coprod_{n\in\omega} \mathbb{R}^n$, including the empty tuple. For a tuple a, write $|a|$ for its length and $a^\frown b$ for concatenation. Define

$$X = \omega \times \mathbb{R}^{<\omega}.$$

An element (k, a), with $a = (a_0, \ldots, a_{n-1})$, is represented by the finite record

$$\mathrm{enc}(k, a) = (n, k, a_0, \ldots, a_{n-1}).$$

The two initial entries are natural numbers stored as real numbers. An entry can be tested for membership in ω by rejecting negative values and repeatedly subtracting 1 until it lies in $[0, 1)$;

*Email: flengyel@gradcenter.cuny.edu

the original entry is a natural number exactly when the remainder is 0. For each fixed $r \geq 1$, an element of X^r is represented by the concatenation of its r records. The length fields determine their boundaries. Computability of partial maps between these sets always refers to these representations.

The natural-number tag will record how a tuple was split. Define the Cantor pairing $c : \omega^2 \to \omega$ by $c(k,l) = (k+l)(k+l+1)/2 + l$. Its inverse is computable by a finite search through successive diagonals of ω^2. Such searches, and integer quotient and remainder by a positive integer, are implementable by BSS loops using addition, subtraction, and comparisons.

Lemma 1.1. *The map $b : X \times X \to X$ defined, for $|a| = n$ and $|a'| = m$, by*

$$b((k,a),(l,a')) = \big((n+m+1)c(k,l) + n,\ a^\frown a'\big)$$

is a bijection, and both b and b^{-1} are total BSS-computable.

Proof. Given $(q,z) \in X$, put $d = |z|$ and compute the unique quotient and remainder $q = (d+1)t + n$, where $0 \leq n \leq d$. Recover $(k,l) = c^{-1}(t)$, split z after its first n entries, and write the resulting tuples as a, a'. Then $b^{-1}(q,z) = ((k,a),(l,a'))$. These operations are inverse to the displayed formula, including when either tuple is empty. Every loop terminates on the indicated natural-number data, and the remaining operations copy finite lists. □

Iterating b gives chosen total computable bijections $b_r : X^r \to X$ for all $r \geq 1$, with $b_1 = 1_X$. The tag is independent of the tuple length; this independence allows every output tag to occur in Lemma 1.1.

2 The category of partial computations

Define $\mathcal{C}_{\mathrm{BSS}}$ to have objects X^r, $r \geq 1$, and all BSS-computable partial functions $f : X^r \rightharpoonup X^s$ as arrows. Equality of arrows means equality of domains and of values on those domains. Composition $g \circ f$ applies f first and is defined exactly where f is defined and g is defined at its value.

A *partial identity* on a subset $S \subseteq A$ is the partial function $A \rightharpoonup A$ with domain S and value a at each $a \in S$. The *restriction* $\bar{f} : A \rightharpoonup A$ of $f : A \rightharpoonup B$ is the partial identity on $\mathrm{dom}(f)$. Thus f is total exactly when $\bar{f} = 1_A$, where 1_A is the identity function on A. Write $0_{A,B} : A \rightharpoonup B$ for the nowhere-defined partial function, whose domain is empty; subscripts are omitted when the source and target are determined by context. For $f : A \rightharpoonup B$ and $g : C \rightharpoonup D$, their *near-product* is the partial function $f \boxtimes g : A \times C \rightharpoonup B \times D$ with value $(f(a), g(c))$ and domain $\mathrm{dom}(f) \times \mathrm{dom}(g)$. On the underlying sets, cartesian products are identified by concatenating the tuples of X-coordinates; thus the near-product of objects is $X^r \boxtimes X^s = X^{r+s}$. The projections $(a,b) \mapsto a$ and $(a,b) \mapsto b$ and the diagonal $a \mapsto (a,a)$ are total.

A *P-category* is a category $\mathcal{K}$ with a functor $\boxtimes : \mathcal{K} \times \mathcal{K} \to \mathcal{K}$, called the near-product, a natural family $\Delta_A : A \to A \boxtimes A$, and projections $\pi_0 : A \boxtimes B \to A$ and $\pi_1 : A \boxtimes B \to B$, subject to the following axioms [5, §5]. For maps $f : T \to A$ and $g : T \to B$, write $\langle f, g \rangle = (f \boxtimes g) \circ \Delta_T$. Object subscripts on projections are suppressed when their types are determined by the formula.

First, $\pi_i \circ \Delta_A = 1_A$ for $i = 0, 1$, and $\langle \pi_0, \pi_1 \rangle = 1_{A \boxtimes B}$. Second, let $\mathcal{P}$ be the smallest subcategory containing all objects, identities, and projections and closed under $\boxtimes$. For every $f : A \to B$ in $\mathcal{K}$ and $p : C \to D$ in $\mathcal{P}$, the projections satisfy

$$\pi_0 \circ (f \boxtimes p) = f \circ \pi_0, \qquad \pi_1 \circ (p \boxtimes f) = f \circ \pi_1.$$

Third, the following families are required to be natural:

$$\alpha_{A,B,C} = \langle \pi_0 \circ \pi_0, \langle \pi_1 \circ \pi_0, \pi_1 \rangle \rangle : (A \boxtimes B) \boxtimes C \longrightarrow A \boxtimes (B \boxtimes C),$$
$$\sigma_{A,B} = \langle \pi_1, \pi_0 \rangle : A \boxtimes B \longrightarrow B \boxtimes A.$$

For an arrow $f : A \to B$ in a P-category, define its restriction by $\bar{f} = \pi_0 \circ \langle 1_A, f \rangle$; the arrow is total when $\bar{f} = 1_A$. In the category of sets and partial functions this agrees with the partial-identity definition above. In this category, α reassociates triples and σ interchanges the two coordinates. The stated axioms hold for the near-product, diagonal, and projections defined above. In particular, every map in $\mathcal{P}$ is total, which gives the two projection equations. A subcategory closed under this structure inherits the axioms.

A family of *zero arrows* consists of one arrow $0_{A,B}$ for each pair of objects, absorbing composition and near-products: every composite or near-product with a zero arrow is the zero arrow of its type. The nowhere-defined partial functions have this property. A P-category with such zero arrows is *prodominical*. A map f is *weakly total* when $f \circ h = 0$ implies $h = 0$ for every composable h. By [5, Proposition 5.1, p. 1261], a prodominical category is *dominical* if every weakly total map is total and $f \boxtimes g = 0$ implies $f = 0$ or $g = 0$. An *isotype* is a category in which all objects are isomorphic.

Proposition 2.1. *The category $\mathcal{C}_{\mathrm{BSS}}$ is a dominical P-category and an isotype.*

Proof. Identities and composition are computable: for a composite, run the first machine and then the second. To compute $\bar{f}$, save the input, run a machine for f, and return the saved input if it halts. To compute $f \boxtimes g$, run the two machines successively on their respective inputs and return both outputs. This halts exactly on $\mathrm{dom}(f) \times \mathrm{dom}(g)$. Projections, diagonals, and the finite coordinate rearrangements are computable by copying records. Thus the P-category structure is inherited from partial functions.

The nowhere-defined map $0_{A,B} : A \rightharpoonup B$ is computed by a nonterminating loop. It absorbs composition and near-products, so $\mathcal{C}_{\mathrm{BSS}}$ is prodominical.

Suppose that $f : A \rightharpoonup B$ is not total, and choose $a \in A \setminus \mathrm{dom}(f)$. The constant map $h : X \to A$ with value a is total and BSS-computable: its program uses the finitely many real entries of a as constants. Since X is nonempty, $h \neq 0$, while $f \circ h = 0$. Hence f is not weakly total. Also, if $f \neq 0$ and $g \neq 0$, choose one point in each domain; their pair belongs to $\mathrm{dom}(f \boxtimes g)$. Heller's two conditions for dominicality follow.

Finally, the maps b_r and their inverses are arrows of $\mathcal{C}_{\mathrm{BSS}}$. They identify every object with X. □

3 Universal evaluation and specialization

In a prodominical isotype, a *Turing morphism* is an arrow $\tau : W \boxtimes A \rightharpoonup B$ such that for every object V and every arrow $\varphi : V \boxtimes A \rightharpoonup B$ there is a total arrow $s : V \to W$ satisfying $\varphi = \tau \circ (s \boxtimes 1_A)$. A *recursion category* is a prodominical isotype with a Turing morphism [5, §9, pp. 1264–1265]. The equality in this definition is equality of partial maps.

We describe the program representation used for the evaluator. A BSS program can be written as a finite instruction list over a fixed set of instruction types: arithmetic, copying, tape movement, conditional branching, and halting. Polynomial and rational instructions are expanded into finite arithmetic instruction lists. The instruction types, register addresses, branch destinations, and list lengths are natural numbers. The real constants are stored separately in a finite table and referred to by their indices. For a finite list $(u_0, \ldots, u_{L-1})$ of natural numbers, put $h_L = 0$, define $h_i = c(u_i, h_{i+1})$ recursively backwards for $0 \leq i < L$, and define its code to be $c(L, h_0)$. To decode, recover L and h_0, unpair L times, and require the final remainder to be 0. Apply this encoding to the integer instruction data. Together with the real constant table, it gives a program code in X.

An interpreter decodes the instruction list, maintains the simulated tape and instruction pointer, and executes one instruction at a time. Integer addresses are handled by finite scans, and a

constant-table lookup copies the indicated real entry. Thus one interpreter performs the indicated computation without placing the simulated constants in its own finite program. Invalid instructions or indices cause it to loop. An output is accepted only if it is a valid record for an element of X. This gives a BSS-computable partial evaluator $U : X \boxtimes X \rightharpoonup X$. It is the usual universal-machine construction of [1, §8, pp. 34–36], with instruction data and real constants stored in separate fields.

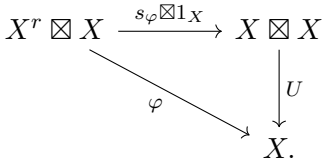
Lemma 3.1. *For every $r \geq 1$ and every BSS-computable partial function $\varphi : X^r \times X \rightharpoonup X$, there is a total BSS-computable $s_\varphi : X^r \to X$ such that, for every $v \in X^r$ and $x \in X$, $U(s_\varphi(v), x)$ and $\varphi(v, x)$ have the same definedness and, when defined, the same value.*

Proof. Fix a program P computing φ on the concatenated input records. Given $v \in X^r$, construct a program whose input is x. Its initialization instructions save x, write the records of v into the parameter part of the simulated input, and append the saved record of x. It then runs P on that input.

To produce this program, copy the finitely many real entries of v to its constant table, together with the constants of P. Generate one finite initialization prefix of the required length, then append the fixed instruction list for P, adjusting addresses and branch destinations. All loops in this construction are bounded by the lengths of the input records or of the generated instruction list. The construction never runs P. It therefore defines a total BSS-computable code map s_φ.

The generated program on input x runs P on (v, x) after a finite initialization. Consequently it halts exactly when $\varphi(v, x)$ is defined and returns the same value. □

The following diagram expresses the specialization equation. Its top arrow is total, and the two paths have the same restriction domain by Lemma 3.1.



Theorem 3.2. *Let $X = \omega \times \coprod_{n \geq 0} \mathbb{R}^n$, with the record representations of Section 1. The category with objects X^r, $r \geq 1$, and all unrestricted-time BSS-computable partial functions between them, allowing finitely many arbitrary real constants in each program, is a dominical recursion category.*

Proof. Proposition 2.1 makes it a dominical isotype. Every object V is X^r for some $r \geq 1$, so Lemma 3.1 says exactly that $U : X \boxtimes X \rightharpoonup X$ is a Turing morphism. □

4 Recovery of fixed-dimensional computations

Let $\mathcal{D}_{\mathrm{BSS}}$ be the category with objects $\mathbb{R}^n$, $n \geq 0$, and all BSS-computable partial functions between them. A *restriction idempotent* is an arrow $e : A \rightharpoonup A$ satisfying $e = \bar{e}$; for partial functions, these are precisely the partial identities. For a category of partial maps $\mathcal{C}$ closed under restrictions, its *restriction idempotent completion* $\mathrm{Split}_{\mathrm{r}}(\mathcal{C})$ has objects (A, e), where $e : A \rightharpoonup A$ is a restriction idempotent belonging to $\mathcal{C}$. An arrow $h : (A, e) \rightharpoonup (B, d)$ is an arrow of $\mathcal{C}$ satisfying $h = d \circ h \circ e$. Its composition is inherited, and the identity of (A, e) is e. This is the splitting construction for restriction idempotents discussed in [3, §2.1, p. 187]. Restrictions are inherited from $\mathcal{C}$. If $\mathcal{C}$ has the near-product of partial functions, define $(A, e) \boxtimes (B, d) = (A \boxtimes B, e \boxtimes d)$ and use the inherited near-product on arrows.

For $n \geq 0$, let $S_n = \{(0, a) \in X : |a| = n\}$, and let $e_n : X \rightharpoonup X$ be the partial identity on S_n. The tag and length tests decide membership in S_n, so e_n is BSS-computable. Define the total encoder $i_n : \mathbb{R}^n \to X$ by $i_n(a) = (0, a)$ and the partial decoder $d_n : X \rightharpoonup \mathbb{R}^n$ by $d_n(0, a) = a$ on S_n. They are BSS-computable and satisfy $d_n \circ i_n = 1_{\mathbb{R}^n}$ and $i_n \circ d_n = e_n$. Their endpoint types here refer to sets and partial functions.

Proposition 4.1. *There is a fully faithful functor $J : \mathcal{D}_{\mathrm{BSS}} \to \mathrm{Split}_{\mathrm{r}}(\mathcal{C}_{\mathrm{BSS}})$ given by*

$$J(\mathbb{R}^n) = (X, e_n), \qquad J(f) = i_m \circ f \circ d_n \quad (f : \mathbb{R}^n \rightharpoonup \mathbb{R}^m).$$

Proof. The map $J(f)$ is BSS-computable, has domain contained in S_n, and takes its values in S_m. Thus $J(f) = e_m \circ J(f) \circ e_n$. Moreover, $J(1_{\mathbb{R}^n}) = e_n$, which is the identity of (X, e_n). For composable f, g, the identity $d_m \circ i_m = 1_{\mathbb{R}^m}$ gives $J(g) \circ J(f) = J(g \circ f)$, including equality of domains.

Conversely, an arrow $h : (X, e_n) \rightharpoonup (X, e_m)$ decodes to the BSS-computable partial function $d_m \circ h \circ i_n$. Encoding and decoding are inverse on these hom-sets. In particular, $i_m \circ (d_m \circ h \circ i_n) \circ d_n = e_m \circ h \circ e_n = h$, and the other composite is the identity by $d_j \circ i_j = 1_{\mathbb{R}^j}$. □

This identifies $\mathcal{D}_{\mathrm{BSS}}$ with the full subcategory on the objects (X, e_n). Moreover, $\overline{J(f)} = J(\bar{f})$, since both sides are the partial identity on $i_n(\mathrm{dom}(f))$. The representation also respects the near-products up to computable isomorphism: on the supported domains, concatenate the real tuples to obtain $S_n \times S_m \cong S_{n+m}$, and split after the first n entries for the inverse. Thus the fixed-dimensional BSS computations are recovered with their partial-map structure.

References

[1] Lenore Blum, Mike Shub, and Steve Smale, On a theory of computation and complexity over the real numbers: NP-completeness, recursive functions and universal machines, *Bulletin of the American Mathematical Society (N.S.)* **21** (1989), no. 1, 1–46. doi:10.1090/S0273-0979-1989-15750-9.

[2] Lenore Blum, Felipe Cucker, Michael Shub, and Steve Smale, *Complexity and Real Computation*, Springer, New York, 1998. doi:10.1007/978-1-4612-0701-6.

[3] J. Robin B. Cockett and Pieter J. W. Hofstra, Introduction to Turing categories, *Annals of Pure and Applied Logic* **156** (2008), nos. 2–3, 183–209. doi:10.1016/j.apal.2008.04.005.

[4] Robert A. Di Paola and Alex Heller, Dominical categories: recursion theory without elements, *Journal of Symbolic Logic* **52** (1987), no. 3, 594–635. doi:10.2307/2274352.

[5] Alex Heller, An existence theorem for recursion categories, *Journal of Symbolic Logic* **55** (1990), no. 3, 1252–1268. doi:10.2307/2274486.

[6] Florian Lengyel, *Recursion categories of coalgebras*, preprint, 2001. arXiv:math/0105256.

[7] Sebastiano Vigna, On the relations between distributive computability and the BSS model, *Theoretical Computer Science* **162** (1996), no. 1, 5–21. doi:10.1016/0304-3975(95)00247-2.